

CS509 - Assignment 3

Purpose. This document defines the algorithms, text-file input formats, expected outputs, and timing requirements for Assignment 3. The driver program must read the input file, prepare the required data structure, call the algorithm, and print both the final result and the algorithm execution time.

Timing rule. Measure only the algorithm execution time. File reading, input parsing, adjacency-list-to-CSR conversion, output printing, and other setup work must not be included.

1. Assignment Structure

Task Type	Algorithms	Work Mode
Individual task	Minimum Spanning Tree (MST): Kruskal's algorithm and Prim's algorithm; weighted undirected CSR graph as input	Individual
Buddy tasks	Gradient Descent and Maxflow-Mincut	Pair of two students

- Programming language: C or C++. C++ is preferred.
- For graph algorithms, the text-file adjacency list must be converted to CSR before the algorithm is called.
- For MST, both Kruskal and Prim must be implemented and run on the same graph inputs so that results and execution times can be compared.

2. Brief Explanation of the Algorithms

2.1 Minimum Spanning Tree (MST)

A Minimum Spanning Tree connects all vertices of a connected, weighted, undirected graph using exactly $V-1$ edges while minimizing the total sum of selected edge weights. The graph must remain connected and contain no cycle in the selected tree.

- Both Kruskal's and Prim's algorithms must compute an MST for the same input graph.
- The total MST weight produced by both algorithms must be the same. If multiple MSTs exist, the selected edge sets may differ while still having the same minimum total weight.

Reference: Princeton Algorithms, Section 4.3 - Minimum Spanning Trees - <https://algs4.cs.princeton.edu/43mst/>

2.1.1 Kruskal's Algorithm

Kruskal's algorithm is a greedy MST algorithm. It considers edges in increasing order of weight and adds an edge only when doing so does not create a cycle. A Disjoint Set Union (DSU/Union-Find) structure is commonly used to detect whether two vertices are already connected.

- Sort all graph edges by non-decreasing weight.
- Process edges in that order and add an edge when its endpoints currently belong to different components.
- Stop after $V-1$ edges have been selected.

2.1.2 Prim's Algorithm

Prim's algorithm is also a greedy MST algorithm. It starts from any vertex and grows one tree by repeatedly selecting the minimum-weight edge that connects a vertex already in the tree to a vertex outside the tree. A min-priority queue is recommended for an efficient implementation.

- Start from a chosen vertex, recommended as vertex 0 for reproducibility.
- Repeatedly select the cheapest edge that expands the current tree to a new vertex.
- Stop when all vertices are included in the tree.

Real-Life Use Cases of MST

- Designing low-cost communication, electrical, road, or pipeline networks that must connect all required locations.
- Network backbone design where total cable or link cost should be minimized.
- Clustering and image-segmentation methods that use spanning-tree structure.
- Approximation and preprocessing steps in routing, layout, and network-design problems.

2.2 CSR Graph

CSR (Compressed Sparse Row) is a compact format used to store sparse graphs efficiently. For a graph, the adjacency list is converted into three arrays:

- `row_ptr`: tells where each vertex's neighbour list starts and ends.
- `col_idx`: stores the neighbouring vertex numbers.
- `values`: stores edge weights for MST, and edge capacities for Maxflow-Mincut.
- CSR applies to MST and Maxflow-Mincut in this assignment. Gradient Descent does not use CSR.

2.3 Gradient Descent

Gradient Descent is an iterative optimization algorithm that repeatedly updates a current value in the direction that decreases an objective function. For this assignment, every student must use the same generic one-variable polynomial representation so that higher-degree functions can be tested without requiring symbolic expression parsing or advanced optimization theory.

```
f(x) = c0 + c1*x + c2*x^2 + ... + cd*x^d
f'(x) = c1 + 2*c2*x + 3*c3*x^2 + ... + d*cd*x^(d-1)
x_new = x - learning_rate * f'(x)
```

- The degree d and coefficients $c_0, c_1, \dots, c_d$ are read from the input file. The same implementation must work for all required polynomial degrees; do not hard-code a separate function for each test case.
- Required tests use polynomial degrees 2, 4, 6, 8 and 10. The supplied functions are convex with a single global minimum, so students are not required to study or prove convexity.
- The input file also supplies the initial x value, learning rate, tolerance and maximum number of iterations. Stop when $|f'(x)| \leq \text{tolerance}$, or when the maximum number of iterations has been reached.
- The program must report the polynomial degree, final x value, $f(x)$, number of iterations, convergence status and algorithm execution time. Increasing the degree increases the arithmetic performed during every iteration and gives additional computation data for the report.

Reference: Stanford CS231n - Loss Functions and Optimization - <https://cs231n.github.io/optimization-1/>

Real-Life Use Cases of Gradient Descent

- Training machine-learning models by minimizing a loss or error function.
- Parameter estimation and curve fitting.

- Engineering design optimization where a differentiable cost function must be minimized.
- Resource-allocation and numerical optimization problems with differentiable objectives.

2.4 Maxflow-Mincut

The Maxflow problem asks for the largest amount of flow that can be sent from a source vertex s to a sink vertex t in a directed graph without exceeding edge capacities. The corresponding minimum s - t cut partitions the vertices into two sets so that s and t are separated and the total capacity of edges crossing from the source side to the sink side is minimum.

- Each directed edge has a non-negative capacity. For the required tests, use positive integer capacities.
- The maximum-flow value must be equal to the minimum-cut capacity.
- After computing maximum flow, the minimum cut can be obtained from the final residual graph by finding all vertices still reachable from the source.
- Any correct maximum-flow implementation may be used. Dinic's algorithm is recommended because the required graph sizes are larger; slower implementations may become impractical on the largest tests.

Reference: MIT OpenCourseWare 6.046J - Max Flow, Min Cut -

<https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2015/resources/lecture-13-incremental-improvement-max-flow-min-cut/>

Real-Life Use Cases of Maxflow-Mincut

- Finding the maximum traffic, data, water, or material that can move through a capacity-limited network.
- Communication-network bandwidth and bottleneck analysis.
- Transportation and logistics planning.
- Image segmentation and computer-vision problems that can be represented as graph cuts.
- Assignment, matching, and scheduling problems that can be reduced to flow networks.

3. General Input-File Rules

- Every test case must be stored in a separate .txt file.
- Use space-separated integer values unless otherwise stated. Gradient Descent parameters may use floating-point values.
- Vertex numbering must be consistent throughout the semester. The recommended convention is 0 to $V-1$.
- For graph inputs, the file must store the graph as an adjacency list.
- The driver reads the adjacency-list file and then calls the provided/helper conversion function to create CSR input before calling the graph algorithm.
- The CSR conversion time must not be included in the reported algorithm execution time.
- The required graph sizes differ per algorithm because of practical time and memory limits; see Section 4.2 for the exact list.
- You are expected to call the CSR conversion function from the previous assignment. DO NOT copy the code into this assignment.
- MST graphs are weighted and undirected, so each edge must appear in the adjacency list of both endpoint vertices. Count each undirected edge once in E .
- Maxflow-Mincut graphs are directed. List only the outgoing edges of each vertex. The input file should contain the original capacity edges; reverse residual edges are created internally by the max-flow implementation.

- MST test graphs must be connected. Maxflow-Mincut test graphs must contain a valid source, sink, and at least one source-to-sink path.

4. CSR Graph

4.1 Graph Input Format and CSR Conversion

- The input format for MST and Maxflow-Mincut is an adjacency list.
- A helper function must be provided to convert the adjacency-list representation into Compressed Sparse Row (CSR) format before the algorithm runs.
- The adjacency-list-to-CSR conversion is preprocessing. Its execution time must not be included in the reported algorithm runtime.
- For an algorithm that uses CSR, timing must begin only after the CSR representation has been prepared.
- Gradient Descent is exempt from CSR conversion because it reads polynomial coefficients and scalar optimization parameters directly from its text file.

4.2 Required Graph Input Sizes

The graph sizes are selected so that the assignment can be run on normal student laptops. Large test graphs should remain sparse. Use the sizes below:

Algorithm	Required Number of Vertices (V)	Note
MST - Kruskal and Prim	10, 100, 10,000, 50,000, 100,000	Use the same graph files for both algorithms. Keep large graphs sparse, approximately $E = 2V$ to $4V$ where practical.
Maxflow-Mincut	10, 100, 1,000, 10,000, 50,000	50,000 is the largest required size. 100,000 vertices is optional. Keep large networks sparse and document E.

- The number of edges (E), graph type, and other relevant properties must also be recorded for each graph test case in the README.
- You can write your own random graph generator file, which takes the number of vertices and edges as input.
- In case on your system, for any graph size, execution is not complete (likely core-dump, out-of-memory, excessive runtime, etc.), please mention that in your report.
- For the optional 100,000-vertex Maxflow-Mincut test, failure is acceptable if the required tests are completed; record the observed behavior instead of hiding it.

4.3 Required Gradient Descent Test Cases

Gradient Descent does not use graph size. To generate uniform report data while also increasing the computation performed per iteration, use five required polynomial tests whose maximum degree increases from 2 to 10. Use the exact functions and parameters listed below.

Test	Degree	Objective function	Initial x	Learning rate	Tolerance	Max iterations	Expected x*
gd_01	2	$x^2 - 6x + 9$	0	0.10	$1e-6$	5,000	3
gd_02	4	$x^4 + 2x^2$	2	0.02	$1e-6$	10,000	0
gd_03	6	$0.1x^6 + 0.5x^4 + x^2$	2	0.02	$1e-6$	20,000	0

Test	Degree	Objective function	Initial x	Learning rate	Tolerance	Max iterations	Expected x*
gd_04	8	$0.01x^8 + 0.05x^6 + 0.2x^4 + x^2$	2	0.01	$1e-8$	50,000	0
gd_05	10	$0.002x^{10} + 0.01x^8 + 0.05x^6 + 0.2x^4 + x^2$	2	0.005	$1e-10$	100,000	0

- Required polynomial degrees are 2, 4, 6, 8 and 10. Coefficients not shown in a function are zero.
- All five required objective functions are convex and have a single global minimum. Students only need to implement polynomial evaluation, derivative evaluation and the Gradient Descent update; no symbolic parser or convexity proof is required.
- Record the polynomial degree, actual number of iterations, final x, final f(x), convergence status and execution time for every test. The expected minimum is provided for correctness checking, but an iteration count is not fixed because small implementation details may change it slightly.
- Because individual runs may still complete very quickly, repeated runs may be used and the average algorithm time may be reported, provided the number of runs is documented. File reading and other setup work must remain outside the timed region.

5. MST Input and Output Test File Format

5.1 Weighted Undirected Adjacency-List File Format

Use the following format for both Kruskal and Prim:

```
V E
u0 degree neighbor1 weight1 neighbor2 weight2 ...
u1 degree neighbor1 weight1 neighbor2 weight2 ...
...
u(V-1) degree neighbor1 weight1 neighbor2 weight2 ...
```

- V: number of vertices. E: number of undirected edges, counting each graph edge once even though it appears in both adjacency lists.
- Every edge must appear in the adjacency list of both endpoint vertices with the same weight.
- The graph must be connected.
- For a vertex with no neighbours, the general format is u 0, but a valid MST input with $V > 1$ cannot contain an isolated vertex.
- The same input file must be used for both Kruskal and Prim.
- MST edge weights may be positive, zero, or negative integers.

5.2 MST Example Input

```
5 7
0 2 1 2 3 6
1 4 0 2 2 3 3 8 4 5
2 2 1 3 4 7
3 3 0 6 1 8 4 9
4 3 1 5 2 7 3 9
```

5.3 Expected MST Output

Print the selected MST edges, total MST weight and execution time for both algorithms. The two implementations must report the same total weight.

```

Algorithm: Kruskal's MST
MST edges:
0 1 2
1 2 3
1 4 5
0 3 6
Total MST weight: 16
Execution time: <value> ms

```

```

Algorithm: Prim's MST
MST edges:
0 1 2
1 2 3
1 4 5
0 3 6
Total MST weight: 16
Execution time: <value> ms

```

- For graphs with multiple valid MSTs, the edge lists may differ. The total weight must still be equal and minimum.

6. Gradient Descent Input and Output Test File Format

6.1 Gradient Descent Text-File Format

```

DEGREE d
COEFFICIENTS c0 c1 c2 ... cd
INITIAL_X x0
LEARNING_RATE alpha
TOLERANCE epsilon
MAX_ITERATIONS n

```

- The objective is $f(x) = c_0 + c_1x + c_2x^2 + \dots + c_dx^d$.
- The COEFFICIENTS line contains exactly $d+1$ floating-point values in ascending power order: constant term first and coefficient of x^d last.
- Compute the derivative from the coefficients: $f'(x) = c_1 + 2c_2x + \dots + dcdx^{(d-1)}$. Students do not need to provide a separate derivative in the input file.
- Update x using $x_{\text{new}} = x - \alpha f'(x)$, where α is the learning rate.
- Stop when $|f'(x)| \leq \epsilon$, or when n iterations have been executed. x_0 is the initial value from which Gradient Descent begins.
- The same polynomial-evaluation and derivative-evaluation code must support every required degree. Do not hard-code the five required objective functions into separate algorithm implementations.

6.2 Gradient Descent Example Input

```

DEGREE 6
COEFFICIENTS 0 0 1 0 0.5 0 0.1
INITIAL_X 2
LEARNING_RATE 0.02
TOLERANCE 0.000001
MAX_ITERATIONS 20000

```

6.3 Expected Gradient Descent Output

```

Algorithm: Gradient Descent
Degree: 6
Final x: approximately 0.000000

```

```

Final f(x): approximately 0.000000
Iterations: <value>
Converged: true
Execution time: <value> ms

```

- Use sufficient floating-point precision in the implementation and output. If the tolerance is not reached before MAX_ITERATIONS, print Converged: false and still report the final x, f(x), iteration count and execution time.

7. Maxflow-Mincut Input and Output Test File Format

7.1 Directed Capacity Adjacency-List File Format

```

V E
u0 degree neighbor1 capacity1 neighbor2 capacity2 ...
u1 degree neighbor1 capacity1 neighbor2 capacity2 ...
...
u(V-1) degree neighbor1 capacity1 neighbor2 capacity2 ...
SOURCE s
SINK t

```

- V: number of vertices. E: number of directed capacity edges.
- Each neighbour is followed by the capacity of the directed edge.
- For required tests, capacities must be positive integers.
- List only outgoing edges in the input file.
- For a vertex with no outgoing edges, write: u 0.
- Do not manually add residual reverse edges with zero capacity; the max-flow implementation creates and updates the residual graph internally.
- s and t must be different valid vertices.

7.2 Maxflow-Mincut Example Input

```

6 10
0 2 1 16 2 13
1 2 2 10 3 12
2 2 1 4 4 14
3 2 2 9 5 20
4 2 3 7 5 4
5 0
SOURCE 0
SINK 5

```

7.3 Expected Maxflow-Mincut Output

Print the maximum-flow value, minimum-cut capacity, the two sides of one valid minimum cut, the cut edges and the algorithm execution time.

```

Algorithm: Maxflow-Mincut
Source: 0
Sink: 5
Maximum flow: 23
Minimum cut capacity: 23
Source side: 0 1 2 4
Sink side: 3 5
Cut edges:
1 3 12
4 3 7
4 5 4
Execution time: <value> ms

```

- If more than one minimum cut exists, the reported partition or cut-edge set may differ. The cut capacity must equal the maximum-flow value.

8. Timing and Measurement Requirements

- Start the timer immediately before calling the algorithm.
- Stop the timer immediately after the algorithm finishes.
- Do not include file reading, input parsing, memory allocation done as setup, adjacency-list-to-CSR conversion, result printing, or file writing.
- Report time in milliseconds or seconds. Use the same unit consistently within a comparison table.
- For very fast inputs, repeated runs may be used and the average may be reported, provided the number of runs is documented.
- Run both MST implementations on the same graph inputs and report their times separately.
- For every MST and Maxflow-Mincut input file, report the algorithm time for that particular graph.
- For every Gradient Descent input file, report the algorithm time and number of iterations.
- For Maxflow-Mincut, the minimum-cut extraction from the final residual graph is considered part of the required algorithm result and should be included in the measured algorithm section.
- For Kruskal, if a sortable edge list is created from the already-prepared CSR inside the Kruskal routine, that extraction and edge sorting are part of Kruskal and must remain inside the timed algorithm call.
- For Maxflow-Mincut, initialization of the residual network from the already-prepared CSR is part of the max-flow algorithm call and must remain inside the timed section.

9. Required Report File with Result Tables

9.1 MST Results Table

File	V	E	Exp. Wt.	Kruskal Wt.	Prim Wt.	Kruskal Time	Prim Time	Equal?	Status
mst_10.txt	10	...	...	...	...	... ms	... ms	Yes/No	Pass/Fail
...	100 / 10000 / 50000 / 100000	...	...	...	...	...	...	...	...

9.2 Gradient Descent Results Table

File	Degree	x0	Rate	Tol.	Max Iter.	Exp. x^*	Exp. $f(x^*)$	Actual x	Actual $f(x)$	Iter. / Time	Status
gd_01.txt	2	0	0.10	1e-6	5,000	3	0	...	...	... / ... ms	Pass/Fail
gd_02.txt - gd_05.txt	4 / 6 / 8 / 10	...	...	...	...	0	0	...	...	... / ... ms	...

9.3 Maxflow-Mincut Results Table

File	V	E	Source	Sink	Exp. Flow	Actual Flow	Cut Capacity	Time	Status
maxflow_10.txt	10	...	0	...	...	...	...	... ms	Pass/Fail
...	100 / 1000 / 10000 / 50000	...	...	...	...	...	...	...	...

10. Suggested Test-File Naming

Algorithm	Suggested Names
MST	mst_10.txt, mst_100.txt, mst_10000.txt, mst_50000.txt, mst_100000.txt
Gradient Descent	gd_01.txt, gd_02.txt, gd_03.txt, gd_04.txt, gd_05.txt
Maxflow-Mincut	maxflow_10.txt, maxflow_100.txt, maxflow_1000.txt, maxflow_10000.txt, maxflow_50000.txt; optional: maxflow_100000.txt

Upload a single report file with tables for each algorithm on Google Classroom.

11. Minimum Expected Driver Behaviour

- Accept the selected algorithm and input-file path through the common wrapper/menu or terminal arguments (built in Assignment 1).
- Read and validate the input file.
- For CSR-based tasks, construct the adjacency list and call the helper function to convert it to CSR.
- Call the selected algorithm.
- For MST, allow Kruskal and Prim to be selected separately and support running both on the same input for comparison.
- Print the final answer in a readable form.
- Print only the measured algorithm execution time as the reported timing.
- Return a clear error message for an invalid or missing input file.
- For invalid Gradient Descent input, reject at minimum an invalid degree, a coefficient count different from $d+1$, a non-positive learning rate, a non-positive tolerance, or a non-positive MAX_ITERATIONS.
- For invalid Maxflow-Mincut input, reject invalid source/sink vertices, source equal to sink, or negative capacities.